

CTE Interface

for S32R41

Rev.0.8.10/16.12.2024

© 2024 NXP Semiconductors

1 Module Index	1
1.1 Software Modules	1
2 Data Structure Index	3
2.1 Data Structures	3
3 Module Documentation	5
3.1 CTE	5
3.1.1 Detailed Description	5
3.2 CTE AUTOSAR Driver Constants	5
3.2.1 Detailed Description	6
3.2.2 Macro Definition Documentation	6
3.2.2.1 CTE_E_HW_ERROR	6
3.2.2.2 CTE_E_PARAM_HANDLE	6
3.2.2.3 CTE_E_PARAM_POINTER	6
3.2.2.4 CTE_E_PARAM_VALUE	7
3.2.2.5 CTE_E_SETUP_FAILED	7
3.2.2.6 CTE_E_WRONG_STATE	7
3.3 CTE AUTOSAR Driver Data Types	7
3.3.1 Detailed Description	8
3.3.2 Typedef Documentation	9
3.3.2.1 Cte_IsrCbType	9
3.3.2.2 Std_ReturnType	9
3.3.3 Enumeration Type Documentation	9
3.3.3.1 Cte_ClockOutStatesType	9
3.3.3.2 Cte_Csi2UnitType	10
3.3.3.3 Cte_Csi2VcType	10
3.3.3.4 Cte_IrqDefinitionType	10
3.3.3.5 Cte_LogicOutStatesType	12
3.3.3.6 Cte_ModeType	12
3.3.3.7 Cte_OutputType	13
3.3.3.8 Cte_OutputTypeType	14
3.3.3.9 Cte_TableUsageType	14
3.3.3.10 Cte_ToggleOutStatesType	15
3.4 CTE AUTOSAR Driver Functions	15
3.4.1 Detailed Description	15
3.4.2 Function Documentation	16
3.4.2.1 Cte_GetLutChecksum()	16
3.4.2.2 Cte_Restart()	16
3.4.2.3 Cte_RfsGenerate()	16
3.4.2.4 Cte_Setup()	16
3.4.2.5 Cte_Start()	17
3.4.2.6 Cte_Stop()	17

3.4.2.7 Cte_UpdateTables()	17
4 Data Structure Documentation	19
4.1 Cte_ActionType Struct Reference	19
4.1.1 Detailed Description	19
4.1.2 Field Documentation	19
4.1.2.1 cteOutputSignalType	19
4.1.2.2 outputSignal	20
4.2 Cte_ModeDefinitionType Struct Reference	20
4.2.1 Detailed Description	20
4.2.2 Field Documentation	20
4.2.2.1 cteWorkingParam0	20
4.2.2.2 cteWorkingParam1	20
4.2.2.3 workingMode	20
4.3 Cte_OutputSignalType Union Reference	21
4.3.1 Field Documentation	21
4.3.1.1 newClockState	21
4.3.1.2 newLogicState	21
4.3.1.3 newToggleState	21
4.4 Cte_SetupParamsType Struct Reference	21
4.4.1 Detailed Description	22
4.4.2 Field Documentation	22
4.4.2.1 cteClockFreq	22
4.4.2.2 cteIrqEvents	22
4.4.2.3 cteMode	22
4.4.2.4 irqExecCore	23
4.4.2.5 irqPriority	23
4.4.2.6 pCteCallback	23
4.4.2.7 repeatCount	23
4.4.2.8 signalDef0Ptr	23
4.4.2.9 signalDef1Ptr	23
4.4.2.10 timeTable0Ptr	24
4.4.2.11 timeTable1Ptr	24
4.5 Cte_SingleOutputDefType Struct Reference	24
4.5.1 Detailed Description	24
4.5.2 Field Documentation	24
4.5.2.1 clockPeriod	24
4.5.2.2 outputSignal	25
4.5.2.3 signalType	25
4.6 Cte_TimeTableDefType Struct Reference	25
4.6.1 Detailed Description	25
4.6.2 Field Documentation	25

4.6.2.1 eventsPtr	25
4.6.2.2 tableLength	26
4.6.2.3 tableTimeExecLimit	26
4.7 Cte_TimingEventType Struct Reference	26
4.7.1 Detailed Description	26
4.7.2 Field Documentation	26
4.7.2.1 absTime	26
4.7.2.2 eventActionsPtr	27
4.8 Cte_WorkingParam0 Union Reference	27
4.8.1 Detailed Description	27
4.8.2 Field Documentation	27
4.8.2.1 cteCsi2Unit	27
4.8.2.2 cteInternalRfsDelay	28
4.9 Cte_WorkingParam1 Union Reference	28
4.9.1 Detailed Description	28
4.9.2 Field Documentation	28
4.9.2.1 cteCsi2Vc	28
4.9.2.2 cteInternalRcsDelay	28
Index	29

Chapter 1

Module Index

1.1 Software Modules

Here is a list of all modules:

CTE	5
CTE AUTOSAR Driver Constants	5
CTE AUTOSAR Driver Data Types	7
CTE AUTOSAR Driver Functions	15

Chapter 2

Data Structure Index

2.1 Data Structures

Here are the data structures with brief descriptions:

Cte_ActionType	Structure for single CTE output event action	19
Cte_ModeDefinitionType	Structure for CTE input source	20
Cte_OutputSignalType		21
Cte_SetupParamsType	The structure for CTE setup	21
Cte_SingleOutputDefType	Structure for CTE output definition	24
Cte_TimeTableDefType	Structure for CTE time table definition	25
Cte_TimingEventType	Structure for CTE time event structure	26
Cte_WorkingParam0	Union type for Mode definitions, first set of parameters	27
Cte_WorkingParam1	Union type for Mode definitions, second set of parameters	28

Chapter 3

Module Documentation

3.1 CTE

CTE.

Modules

- [CTE AUTOSAR Driver Constants](#)
CTE Driver Constants .
- [CTE AUTOSAR Driver Data Types](#)
CTE Driver Data Types .
- [CTE AUTOSAR Driver Functions](#)
CTE Driver Functions .

3.1.1 Detailed Description

CTE.

Describes the functions, data types and structures that make up the CTE AUTOSAR Complex Device Driver user interface. Proper usage of the CSI2 Driver is described in CTE User Guide, in RSDK User Manual.

3.2 CTE AUTOSAR Driver Constants

CTE Driver Constants .

Macros

- #define `CTE_E_PARAM_POINTER` ((uint8)0x01U)
API request called with an invalid parameter (Nullpointer).
- #define `CTE_E_PARAM_VALUE` ((uint8)0x02U)
API request called with invalid parameter (invalid value).
- #define `CTE_E_PARAM_HANDLE` ((uint8)0x03U)
API request called with invalid parameter (out of range).
- #define `CTE_E_SETUP_FAILED` ((uint8)0x04U)
Setup of Cte Driver failed.
- #define `CTE_E_WRONG_STATE` ((uint8)0x05U)
Incorrect driver status.
- #define `CTE_E_HW_ERROR` ((uint8)0x06U)
Hardware error.

3.2.1 Detailed Description

CTE Driver Constants .

3.2.2 Macro Definition Documentation

3.2.2.1 CTE_E_HW_ERROR

```
#define CTE_E_HW_ERROR ((uint8)0x06U)
```

Hardware error.

3.2.2.2 CTE_E_PARAM_HANDLE

```
#define CTE_E_PARAM_HANDLE ((uint8)0x03U)
```

API request called with invalid parameter (out of range).

3.2.2.3 CTE_E_PARAM_POINTER

```
#define CTE_E_PARAM_POINTER ((uint8)0x01U)
```

API request called with an invalid parameter (Nullpointer).

3.2.2.4 CTE_E_PARAM_VALUE

```
#define CTE_E_PARAM_VALUE ((uint8)0x02U)
```

API request called with invalid parameter (invalid value).

3.2.2.5 CTE_E_SETUP_FAILED

```
#define CTE_E_SETUP_FAILED ((uint8)0x04U)
```

Setup of Cte Driver failed.

3.2.2.6 CTE_E_WRONG_STATE

```
#define CTE_E_WRONG_STATE ((uint8)0x05U)
```

Incorrect driver status.

3.3 CTE AUTOSAR Driver Data Types

CTE Driver Data Types .

Data Structures

- union [Cte_WorkingParam0](#)
Union type for Mode definitions, first set of parameters.
- union [Cte_WorkingParam1](#)
Union type for Mode definitions, second set of parameters.
- struct [Cte_ModeDefinitionType](#)
Structure for CTE input source.
- struct [Cte_SingleOutputDefType](#)
Structure for CTE output definition.
- union [Cte_OutputSignalType](#)
- struct [Cte_ActionType](#)
Structure for single CTE output event action.
- struct [Cte_TimingEventType](#)
Structure for CTE time event structure.
- struct [Cte_TimeTableDefType](#)
Structure for CTE time table definition.
- struct [Cte_SetupParamsType](#)
The structure for CTE setup.

Typedefs

- typedef rsdkStatus_t [Std_ReturnType](#)
- typedef void(* [Cte_IsrCbType](#)) (uint32 cteIrqReport)

Definition of callback function type to be called by the CTE interrupt handler.

Enumerations

- enum [Cte_ModeType](#) { [CTE_MASTER](#) = 0u , [CTE_SLAVE_EXTERNAL](#) , [CTE_SLAVE_CSI2](#) , [CTE_MODE_MAX](#) }
CTE input mode.
- enum [Cte_Csi2UnitType](#) { [CTE_CSI2_UNIT_0](#) = 0u , [CTE_CSI2_UNIT_1](#) , [CTE_CSI2_UNIT_MAX](#) }
MIPI-CSI2 units enumeration as CTE input source.
- enum [Cte_Csi2VcType](#) { [CTE_CSI2_VC_0](#) = 0u , [CTE_CSI2_VC_1](#) , [CTE_CSI2_VC_2](#) , [CTE_CSI2_VC_3](#) , [CTE_CSI2_VC_MAX](#) }
MIPI-CSI2 virtual channels enumeration as CTE input source.
- enum [Cte_OutputType](#) { [CTE_OUTPUT_SPT_0](#) = 0u , [CTE_OUTPUT_SPT_1](#) , [CTE_OUTPUT_SPT_2](#) , [CTE_OUTPUT_SPT_3](#) , [CTE_OUTPUT_CTEP_0](#) , [CTE_OUTPUT_CTEP_1](#) , [CTE_OUTPUT_CTEP_2](#) , [CTE_OUTPUT_CTEP_3](#) , [CTE_OUTPUT_CTEP_4](#) , [CTE_OUTPUT_CTEP_5](#) , [CTE_OUTPUT_CTEP_6](#) , [CTE_OUTPUT_CTEP_7](#) , [CTE_OUTPUT_SPT_RCS](#) , [CTE_OUTPUT_SPT_RFS](#) , [CTE_OUTPUT_FLEX_0](#) , [CTE_OUTPUT_FLEX_1](#) , [CTE_OUTPUT_MAX](#) }
CTE available output signals.
- enum [Cte_OutputTypeType](#) { [CTE_OUT_HIZ](#) = 0u , [CTE_OUT_TOGGLE](#) , [CTE_OUT_CLOCK](#) , [CTE_OUT_LOGIC](#) , [CTE_OUT_MAX](#) }
CTE signal output types.
- enum [Cte_ToggleOutStatesType](#) { [CTE_TOGGLE_SET_TO_LOW](#) = 0u , [CTE_TOGGLE_SET_TO_HIGH](#) , [CTE_TOGGLE_FLIP](#) , [CTE_TOGGLE_DONT_CARE](#) }
CTE signal output state for TOGGLE type.
- enum [Cte_LogicOutStatesType](#) { [CTE_LOGIC_SET_TO_LOW](#) = 0u , [CTE_LOGIC_SET_TO_HIGH](#) , [CTE_LOGIC_SET_TO_HIGH_Z](#) , [CTE_LOGIC_UNCHANGED](#) }
CTE signal output state for LOGIC type.
- enum [Cte_ClockOutStatesType](#) { [CTE_CLOCK_SET_TO_LOW](#) = 0u , [CTE_CLOCK_ACTIVE_SYNC](#) , [CTE_CLOCK_ACTIVE](#) , [CTE_CLOCK_SET_TO_HIGH](#) }
CTE signal output state for CLOCK type.
- enum [Cte_TableUsageType](#) { [CTE_ONE_TABLE](#) = 0u , [CTE_TWO_TABLES](#) }
CTE table usage.
- enum [Cte_IrqDefinitionType](#) { [CTE_IRQ_TT0_START](#) = 0x01u , [CTE_IRQ_TT1_START](#) = 0x02u , [CTE_IRQ_TT0_END](#) = 0x04u , [CTE_IRQ_TT1_END](#) = 0x08u , [CTE_IRQ_RCS](#) = 0x40u , [CTE_IRQ_RFS](#) = 0x80u , [CTE_IRQ_TABLE_EXEC_END](#) = 0x200u }
The enumeration of the available CTE interrupts.

3.3.1 Detailed Description

CTE Driver Data Types .

3.3.2 Typedef Documentation

3.3.2.1 Cte_IsrCbType

```
typedef void(* Cte_IsrCbType) (uint32 cteIrqReport)
```

Definition of callback function type to be called by the CTE interrupt handler.

This callback must be used for processing a CTE event interrupt.

Simple callback function definition example : void CallbackCte(Cte_IrqDefinitionType irqs) { ... }

Parameters

in	<i>cteIrqReport</i>	- a combination of Cte_IrqDefinitionType elements
----	---------------------	---

Returns

nothing

3.3.2.2 Std_ReturnType

```
typedef rsdkStatus_t Std_ReturnType
```

3.3.3 Enumeration Type Documentation

3.3.3.1 Cte_ClockOutStatesType

```
enum Cte_ClockOutStatesType
```

CTE signal output state for CLOCK type.

The only possible states for CLOCK type.

Enumerator

CTE_CLOCK_SET_TO_LOW	Keep the CLOCK output to low
CTE_CLOCK_ACTIVE_SYNC	CLOCK output active and synchronized on rising edge
CTE_CLOCK_ACTIVE	CLOCK output
CTE_CLOCK_SET_TO_HIGH	Keep the CLOCK output at high level

3.3.3.2 Cte_Csi2UnitType

enum `Cte_Csi2UnitType`

MIPI-CSI2 units enumeration as CTE input source.

Used only in CTE_INPUT_CSI2/Master mode. As the driver is intended to be used for S32R294 platform too, a platform differentiation must be done, as S32R294 has two CSI2 units and S32R45 has four.

Enumerator

CTE_CSI2_UNIT_0	MIPI-CSI2 unit 0
CTE_CSI2_UNIT_1	MIPI-CSI2 unit 1
CTE_CSI2_UNIT_MAX	MIPI-CSI2 unit limit (not used)

3.3.3.3 Cte_Csi2VcType

enum `Cte_Csi2VcType`

MIPI-CSI2 virtual channels enumeration as CTE input source.

Used only in CTE_INPUT_CSI2/Master mode.

Enumerator

CTE_CSI2_VC_0	MIPI-CSI2 virtual channel 0
CTE_CSI2_VC_1	MIPI-CSI2 virtual channel 1
CTE_CSI2_VC_2	MIPI-CSI2 virtual channel 2
CTE_CSI2_VC_3	MIPI-CSI2 virtual channel 3
CTE_CSI2_VC_MAX	MIPI-CSI2 virtual channel limit (not used)

3.3.3.4 Cte_IrqDefinitionType

enum `Cte_IrqDefinitionType`

The enumeration of the available CTE interrupts.

These are all available interrupts which the CTE can generate. Only one interrupt callback is used by the driver, but any combination of these signals can be used.

Enumerator

CTE_IRQ_TT0_START	Start execution of TimeTable 0
CTE_IRQ_TT1_START	Start execution of TimeTable 1
CTE_IRQ_TT0_END	End of TimeTable 0 execution
CTE_IRQ_TT1_END	End of TimeTable 1 execution
CTE_IRQ_RCS	Raising edge of RCS signal
CTE_IRQ_RFS	Raising edge of RFS signal
CTE_IRQ_TABLE_EXEC_END	Table execution was finished

3.3.3.5 Cte_LogicOutStatesType

enum `Cte_LogicOutStatesType`

CTE signal output state for LOGIC type.

The only possible states for LOGIC type.

Enumerator

CTE_LOGIC_SET_TO_LOW	Set the LOGIC output to low
CTE_LOGIC_SET_TO_HIGH	Set the LOGIC output to high
CTE_LOGIC_SET_TO_HIGH↔ _Z	Set the LOGIC output to High-Z
CTE_LOGIC_UNCHANGED	Set the LOGIC output to High-Z

3.3.3.6 Cte_ModeType

enum `Cte_ModeType`

CTE input mode.

Defines the two possible CTE working modes : using internal CSI2 signals (Master mode) or external PAD signals (Slave mode)

Enumerator

CTE_MASTER	CTE in Master mode. This means the timing is decided only by software control, table execution start or reset.
CTE_SLAVE_EXTERNAL	CTE in Slave mode, table execution triggered from PADs (check IOMUX excel file for RCS_I and/or RFS_I)
CTE_SLAVE_CSI2	CTE in Slave mode, table execution triggered by MIPI-CSI2 RFS/RCS signals.
CTE_MODE_MAX	CTE input mode limit, not to be used

3.3.3.7 Cte_OutputType

enum `Cte_OutputType`

CTE available output signals.

All possible CTE output signals. For normal usage only a subset can be used. Each output must be used separately, no combinations are permitted.

Enumerator

CTE_OUTPUT_SPT_0	CTE SPT event 0 is the output
CTE_OUTPUT_SPT_1	CTE SPT event 1 is the output
CTE_OUTPUT_SPT_2	CTE SPT event 2 is the output
CTE_OUTPUT_SPT_3	CTE SPT event 3 is the output
CTE_OUTPUT_CTEP_0	CTE PAD_0 is the output
CTE_OUTPUT_CTEP_1	CTE PAD_1 is the output
CTE_OUTPUT_CTEP_2	CTE PAD_2 is the output
CTE_OUTPUT_CTEP_3	CTE PAD_3 is the output
CTE_OUTPUT_CTEP_4	CTE PAD_4 is the output
CTE_OUTPUT_CTEP_5	CTE PAD_5 is the output
CTE_OUTPUT_CTEP_6	CTE PAD_6 is the output
CTE_OUTPUT_CTEP_7	CTE PAD_7 is the output
CTE_OUTPUT_SPT_RCS	CTE SPT RCS event is the output
CTE_OUTPUT_SPT_RFS	CTE SPT RFS event is the output
CTE_OUTPUT_FLEX_0	CTE Flextimer event 0 is the output

Enumerator

CTE_OUTPUT_FLEX_1	CTE Flextimer event 1 is the output
CTE_OUTPUT_MAX	CTE output definitions limit (not used)

3.3.3.8 Cte_OutputTypeType

enum `Cte_OutputTypeType`

CTE signal output types.

The possible different usage types for the CTE outputs. CTE_OUTPUT_SPT_x events can be only:

- LOGIC and only TO_LOW or TO_HIGH states to be used.
- TOGGLE, which will work as PULSE; only LOW and HIGH states to be used too CTE_OUTPUT_FLEX_x events can be only LOGIC and only TO_LOW or TO_HIGH states to be used.

Enumerator

CTE_OUT_HIZ	Output type is only High-Z (if possible)
CTE_OUT_TOGGLE	Output type is toggle, low/hi/toggle
CTE_OUT_CLOCK	Output type is clock, low/hi/toggle with[out] sync
CTE_OUT_LOGIC	Output type is logic, low/high/Hi-Z levels
CTE_OUT_MAX	Output type limit (not used)

3.3.3.9 Cte_TableUsageType

enum `Cte_TableUsageType`

CTE table usage.

This is the user intended usage for the CTE driver/timing table. The driver implement this as necessary.

Enumerator

CTE_ONE_TABLE	Only one table, the same for all chirps
CTE_TWO_TABLES	Two tables, first for even chirps and second for odds

3.3.3.10 Cte_ToggleOutStatesType

enum `Cte_ToggleOutStatesType`

CTE signal output state for TOGGLE type.

The only possible states for TOGGLE type.

Enumerator

CTE_TOGGLE_SET_TO_LOW	Set the TOGGLE output to low
CTE_TOGGLE_SET_TO_HIGH	Set the TOGGLE output to high
CTE_TOGGLE_FLIP	Flip the TOGGLE output (high->low; low->high)
CTE_TOGGLE_DONT_CARE	The TOGGLE output remains the same

3.4 CTE AUTOSAR Driver Functions

CTE Driver Functions .

Functions

- `Std_ReturnType Cte_Setup` (const `Cte_SetupParamsType` *cteInitParamsPtr, uint64 *lutChecksumPtr)
Initialization procedure for CTE driver.
- `Std_ReturnType Cte_Start` (void)
Start procedure for CTE.
- `Std_ReturnType Cte_Stop` (void)
Stop procedure for CTE.
- `Std_ReturnType Cte_Restart` (void)
Restart CTE.
- `Std_ReturnType Cte_RfsGenerate` (void)
Generate a RFS software signal.
- `Std_ReturnType Cte_UpdateTables` (`Cte_TimeTableDefType` *table0Ptr, `Cte_TimeTableDefType` *table1Ptr, uint64 *lutChecksumPtr)
Procedure to update only the existing timing tables.
- uint64 `Cte_GetLutChecksum` (void)
Get the checksum of the timing LUT.

3.4.1 Detailed Description

CTE Driver Functions .

3.4.2 Function Documentation

3.4.2.1 Cte_GetLutChecksum()

```
uint64 Cte_GetLutChecksum (
    void )
```

Get the checksum of the timing LUT.

This procedure returns the current checksum reported by the hardware, only 40 bits. The value can be compared to the previous values.

Returns

LUT checksum

3.4.2.2 Cte_Restart()

```
Std_ReturnType Cte_Restart (
    void )
```

Restart CTE.

This procedure is a single call for Cte_Stop and a Cte_Start.

Returns

E_OK/RSDK_SUCCESS = restart succeeded

other values = restart failed, use the appropriate tools to detect the issue

3.4.2.3 Cte_RfsGenerate()

```
Std_ReturnType Cte_RfsGenerate (
    void )
```

Generate a RFS software signal.

The procedure can be used only in Slave mode, to reset the time table execution. The real CTE execution must be triggered by a RCS signal.

Returns

E_OK/RSDK_SUCCESS = call succeeded

other values = call failed, use the appropriate tools to detect the issue

3.4.2.4 Cte_Setup()

```
Std_ReturnType Cte_Setup (
    const Cte_SetupParamsType * cteInitParamsPtr,
    uint64 * lutChecksumPtr )
```

Initialization procedure for CTE driver.

After initialization the CTE is not started, a specific Cte_Start call must be used for this. The operation can be done at any moment; if the CTE is working, it will be stopped.

Parameters

in	<i>cteInitParamsPtr</i>	= pointer to the initialization structure
in	<i>lutChecksumPtr</i>	= pointer to a uint64 value, which will receive the final LUT checksum; this value can be checked later using Cte_GetLutChecksum

Returns

E_OK/RSDK_SUCCESS = initialization succeeded
 other values = initialization failed, use the appropriate tools to detect the issue

3.4.2.5 Cte_Start()

```
Std_ReturnType Cte_Start (
    void )
```

Start procedure for CTE.

After this call the CTE will start to work, if successful. The procedure could be unsuccessful only if the driver was not initialized before. The exit status can be different than E_OK/RSDK_SUCCESS; if the unit is running, the return is not OK but it is only a warning, signaling the CTE was already working and this was not interrupt. Start after a Stop will use the initialization done before.

Returns

E_OK/RSDK_SUCCESS = start succeeded
 other values = start failed, use the appropriate tools to detect the issue

3.4.2.6 Cte_Stop()

```
Std_ReturnType Cte_Stop (
    void )
```

Stop procedure for CTE.

After this call the CTE will stop to work, if successful. The procedure could be unsuccessful only if the driver is not working at the call time.

Returns

E_OK/RSDK_SUCCESS = stop succeeded
 other values = stop failed, use the appropriate tools to detect the issue

3.4.2.7 Cte_UpdateTables()

```
Std_ReturnType Cte_UpdateTables (
    Cte_TimeTableDefType * table0Ptr,
    Cte_TimeTableDefType * table1Ptr,
    uint64 * lutChecksumPtr )
```

Procedure to update only the existing timing tables.

The procedure can be used only after a previous successful CTE initialization. If the CTE is working, it will be stopped and restarted after table changed. If stopped, it will remains in the same state. It is recommendable to do like this.

Parameters

in	<i>table0Ptr, table1Ptr</i>	= pointer to the new table(s); first pointer must not be NULL; if second is NULL, only one table used, else two tables used
in	<i>lutChecksumPtr</i>	= pointer to a uint64 value, which will receive the final LUT checksum; this value can be checked later using Cte_GetLutChecksum

Returns

E_OK/RSDK_SUCCESS = initialization succeeded

other values = initialization failed, use the appropriate tools to detect the issue

Chapter 4

Data Structure Documentation

4.1 Cte_ActionType Struct Reference

Structure for single CTE output event action.

```
#include <Cte_Types.h>
```

Data Fields

- [Cte_OutputType](#) outputSignal
- [Cte_OutputSignalType](#) cteOutputSignalType

4.1.1 Detailed Description

Structure for single CTE output event action.

The new state will be interpreted according to the defined type of the output signal. This structure define a single output change.

4.1.2 Field Documentation

4.1.2.1 cteOutputSignalType

[Cte_OutputSignalType](#) cteOutputSignalType

4.1.2.2 outputSignal

`Cte_OutputType` outputSignal

The specific output signal which must change the state

4.2 Cte_ModeDefinitionType Struct Reference

Structure for CTE input source.

```
#include <Cte_Types.h>
```

Data Fields

- `Cte_ModeType` workingMode
- `Cte_WorkingParam0` cteWorkingParam0
- `Cte_WorkingParam1` cteWorkingParam1

4.2.1 Detailed Description

Structure for CTE input source.

CTE_clock refers to CTE input clock, before any internal divisor, i.e. MC_CGM_0(PER_CLK) for S32R45

4.2.2 Field Documentation

4.2.2.1 cteWorkingParam0

`Cte_WorkingParam0` cteWorkingParam0

4.2.2.2 cteWorkingParam1

`Cte_WorkingParam1` cteWorkingParam1

4.2.2.3 workingMode

`Cte_ModeType` workingMode

The source of input trigger signal

4.3 Cte_OutputSignalType Union Reference

```
#include <Cte_Types.h>
```

Data Fields

- [Cte_ToggleOutStatesType newToggleState](#)
- [Cte_LogicOutStatesType newLogicState](#)
- [Cte_ClockOutStatesType newClockState](#)

4.3.1 Field Documentation

4.3.1.1 newClockState

```
Cte_ClockOutStatesType newClockState
```

New state for CLOCK output

4.3.1.2 newLogicState

```
Cte_LogicOutStatesType newLogicState
```

New state for LOGIC output

4.3.1.3 newToggleState

```
Cte_ToggleOutStatesType newToggleState
```

New state for TOGGLE output

4.4 Cte_SetupParamsType Struct Reference

The structure for CTE setup.

```
#include <Cte_Types.h>
```

Data Fields

- [Cte_ModeDefinitionType](#) `cteMode`
- `uint32` `cteClockFreq`
- `uint16` `repeatCount`
- [Cte_SingleOutputDefType](#) * `signalDef0Ptr`
- [Cte_SingleOutputDefType](#) * `signalDef1Ptr`
- [Cte_TimeTableDefType](#) * `timeTable0Ptr`
- [Cte_TimeTableDefType](#) * `timeTable1Ptr`
- [Cte_IrqDefinitionType](#) `cteIrqEvents`
- [Cte_IsrCbType](#) `pCteCallback`
- `int8` `irqExecCore`
- `uint8` `irqPriority`

4.4.1 Detailed Description

The structure for CTE setup.

All necessary parameters to initialize the CTE hardware.

4.4.2 Field Documentation

4.4.2.1 `cteClockFreq`

```
uint32 cteClockFreq
```

The frequency of the CTE_clock signal, in Hz, at working time

4.4.2.2 `cteIrqEvents`

```
Cte_IrqDefinitionType cteIrqEvents
```

The requested combination of interrupt sequence to be used

4.4.2.3 `cteMode`

```
Cte_ModeDefinitionType cteMode
```

Specify the CTE usage, as Master (input from MIPI-CSI2 RCS/RFS) or Slave (input from external RCS/RFS)

4.4.2.4 irqExecCore

int8 irqExecCore

Processor core to execute the irq code. Usually the current core.

4.4.2.5 irqPriority

uint8 irqPriority

Priority for the interrupt request execution

4.4.2.6 pCteCallback

Cte_IsrCbType pCteCallback

The application callback to be used for the requested events

4.4.2.7 repeatCount

uint16 repeatCount

The number of times the table execution will be repeated until going to HALT state. 0 (zero) means "forever". If two alternative tables used, each table execution will decrease the counter.

4.4.2.8 signalDef0Ptr

Cte_SingleOutputDefType* signalDef0Ptr

Pointer to the output signals working mode definitions. Undefined outputs will be set as HiZ. All used SPT events used must be defined, using the same output type. All used FLEX line used must be defined, using the same output type. The table end is signaled using a not defined output (please use CTE_OUTPUT_MAX). This table contains the definitions used for Table0 execution.

4.4.2.9 signalDef1Ptr

Cte_SingleOutputDefType* signalDef1Ptr

Pointer to the output signals working mode definitions. Similar to pSignalDef0, but this table contains the definitions to be used for Table1 execution.

4.4.2.10 timeTable0Ptr

```
Cte_TimeTableDefType* timeTable0Ptr
```

Pointer for the first table to be used. Must be not NULL.

4.4.2.11 timeTable1Ptr

```
Cte_TimeTableDefType* timeTable1Ptr
```

Pointer for the second table to be used. If NULL, two alternative tables mode will be used.

4.5 Cte_SingleOutputDefType Struct Reference

Structure for CTE output definition.

```
#include <Cte_Types.h>
```

Data Fields

- [Cte_OutputType](#) outputSignal
- [Cte_OutputTypeType](#) signalType
- uint32 [clockPeriod](#)

4.5.1 Detailed Description

Structure for CTE output definition.

Each used output must be defined into one structure like this.

4.5.2 Field Documentation

4.5.2.1 clockPeriod

```
uint32 clockPeriod
```

The clock period, in ns, used only for CLOCK type

4.5.2.2 outputSignal

`Cte_OutputType` outputSignal

The referred output signal

4.5.2.3 signalType

`Cte_OutputTypeType` signalType

The type selected for the specified signal

4.6 Cte_TimeTableDefType Struct Reference

Structure for CTE time table definition.

```
#include <Cte_Types.h>
```

Data Fields

- uint8 `tableLength`
- uint32 `tableTimeExecLimit`
- `Cte_TimingEventType` * `eventsPtr`

4.6.1 Detailed Description

Structure for CTE time table definition.

This is a structure for only one table. If two tables are used, two structures must be defined.

4.6.2 Field Documentation

4.6.2.1 eventsPtr

`Cte_TimingEventType`* eventsPtr

Pointer to the events list, which must have the tableLength corrected defined events, or an error will be reported. A NULL value for pEventActions before the tableLength limit will be used as a pEvents limit and the TableLength will be truncated to the counted number, with no errors. This means is possible to use a big number for tableLength and define the table end using a NULL pointer for pEventActions.

4.6.2.2 tableLength

```
uint8 tableLength
```

The number of defined events to be used/added. It must be up to 32 for two tables usage and up to 64 if single table used. A zero length means the table is not well defined and an error will be returned.

4.6.2.3 tableTimeExecLimit

```
uint32 tableTimeExecLimit
```

The limited time for table processing, in ns. 0 means no table end on time limit.

4.7 Cte_TimingEventType Struct Reference

Structure for CTE time event structure.

```
#include <Cte_Types.h>
```

Data Fields

- uint32 [absTime](#)
- [Cte_ActionType](#) * [eventActionsPtr](#)

4.7.1 Detailed Description

Structure for CTE time event structure.

For each changing time moment at least one event must be added. For the same time many timing events can be specified, according to the desired signal changes, but not necessary to specify all active signals. If the output is defined/active but not changed at a time, the previous state will be maintained.

4.7.2 Field Documentation

4.7.2.1 absTime

```
uint32 absTime
```

The absolute time of the event, in ns, relative to trigger command

4.7.2.2 eventActionsPtr

`Cte_ActionType* eventActionsPtr`

Pointer to the actions to be done at the specified time. The actions list must be :

- not empty
- not more than one action for one output
- no other outputs than the ones defined or next point will be applied
- the end of the list is signaled by an output not defined in Cte_OutputType as normal output (please use CTE_OUTPUT_MAX).

4.8 Cte_WorkingParam0 Union Reference

Union type for Mode definitions, first set of parameters.

```
#include <Cte_Types.h>
```

Data Fields

- `Cte_Csi2UnitType cteCsi2Unit`
- `uint8 cteInternalRfsDelay`

4.8.1 Detailed Description

Union type for Mode definitions, first set of parameters.

First parameter is for internal CSI2 signals, the second is for external signal

4.8.2 Field Documentation

4.8.2.1 cteCsi2Unit

`Cte_Csi2UnitType cteCsi2Unit`

MIPI-CSI2 unit to be used for input trigger. Value used only for CTE_SLAVE_CSI2 mode.

4.8.2.2 cteInternalRfsDelay

```
uint8 cteInternalRfsDelay
```

Internal RFS delay, from detecting to usage, in CTE_clock ticks, values in [0...15] interval only. Used/checked only for CTE_SLAVE_EXTERNAL mode.

4.9 Cte_WorkingParam1 Union Reference

Union type for Mode definitions, second set of parameters.

```
#include <Cte_Types.h>
```

Data Fields

- [Cte_Csi2VcType](#) cteCsi2Vc
- [uint8](#) cteInternalRcsDelay

4.9.1 Detailed Description

Union type for Mode definitions, second set of parameters.

First parameter is for internal CSI2 signals, the second is for external signal

4.9.2 Field Documentation

4.9.2.1 cteCsi2Vc

```
Cte\_Csi2VcType cteCsi2Vc
```

MIPI-CSI2 Virtual Channel to be used for input trigger. Value used only for CTE_SLAVE_CSI2 mode.

4.9.2.2 cteInternalRcsDelay

```
uint8 cteInternalRcsDelay
```

Internal RCS delay, from detecting to usage, in CTE_clock ticks, values in [0...15] interval only. Used/checked only for CTE_SLAVE_EXTERNAL source.

Index

- absTime
 - Cte_TimingEventType, [26](#)
- clockPeriod
 - Cte_SingleOutputDefType, [24](#)
- CTE, [5](#)
- CTE AUTOSAR Driver Constants, [5](#)
 - CTE_E_HW_ERROR, [6](#)
 - CTE_E_PARAM_HANDLE, [6](#)
 - CTE_E_PARAM_POINTER, [6](#)
 - CTE_E_PARAM_VALUE, [6](#)
 - CTE_E_SETUP_FAILED, [7](#)
 - CTE_E_WRONG_STATE, [7](#)
- CTE AUTOSAR Driver Data Types, [7](#)
 - CTE_CLOCK_ACTIVE, [9](#)
 - CTE_CLOCK_ACTIVE_SYNC, [9](#)
 - CTE_CLOCK_SET_TO_HIGH, [9](#)
 - CTE_CLOCK_SET_TO_LOW, [9](#)
 - Cte_ClockOutStatesType, [9](#)
 - CTE_CSI2_UNIT_0, [10](#)
 - CTE_CSI2_UNIT_1, [10](#)
 - CTE_CSI2_UNIT_MAX, [10](#)
 - CTE_CSI2_VC_0, [10](#)
 - CTE_CSI2_VC_1, [10](#)
 - CTE_CSI2_VC_2, [10](#)
 - CTE_CSI2_VC_3, [10](#)
 - CTE_CSI2_VC_MAX, [10](#)
 - Cte_Csi2UnitType, [10](#)
 - Cte_Csi2VcType, [10](#)
 - CTE_IRQ_RCS, [12](#)
 - CTE_IRQ_RFS, [12](#)
 - CTE_IRQ_TABLE_EXEC_END, [12](#)
 - CTE_IRQ_TT0_END, [12](#)
 - CTE_IRQ_TT0_START, [12](#)
 - CTE_IRQ_TT1_END, [12](#)
 - CTE_IRQ_TT1_START, [12](#)
 - Cte_IrqDefinitionType, [10](#)
 - Cte_IsrCbType, [9](#)
 - CTE_LOGIC_SET_TO_HIGH, [12](#)
 - CTE_LOGIC_SET_TO_HIGH_Z, [12](#)
 - CTE_LOGIC_SET_TO_LOW, [12](#)
 - CTE_LOGIC_UNCHANGED, [12](#)
 - Cte_LogicOutStatesType, [12](#)
 - CTE_MASTER, [13](#)
 - CTE_MODE_MAX, [13](#)
 - Cte_ModeType, [12](#)
 - CTE_ONE_TABLE, [14](#)
 - CTE_OUT_CLOCK, [14](#)
 - CTE_OUT_HIZ, [14](#)
 - CTE_OUT_LOGIC, [14](#)
 - CTE_OUT_MAX, [14](#)
 - CTE_OUT_TOGGLE, [14](#)
 - CTE_OUTPUT_CTEP_0, [13](#)
 - CTE_OUTPUT_CTEP_1, [13](#)
 - CTE_OUTPUT_CTEP_2, [13](#)
 - CTE_OUTPUT_CTEP_3, [13](#)
 - CTE_OUTPUT_CTEP_4, [13](#)
 - CTE_OUTPUT_CTEP_5, [13](#)
 - CTE_OUTPUT_CTEP_6, [13](#)
 - CTE_OUTPUT_CTEP_7, [13](#)
 - CTE_OUTPUT_FLEX_0, [13](#)
 - CTE_OUTPUT_FLEX_1, [14](#)
 - CTE_OUTPUT_MAX, [14](#)
 - CTE_OUTPUT_SPT_0, [13](#)
 - CTE_OUTPUT_SPT_1, [13](#)
 - CTE_OUTPUT_SPT_2, [13](#)
 - CTE_OUTPUT_SPT_3, [13](#)
 - CTE_OUTPUT_SPT_RCS, [13](#)
 - CTE_OUTPUT_SPT_RFS, [13](#)
 - Cte_OutputType, [13](#)
 - Cte_OutputTypeType, [14](#)
 - CTE_SLAVE_CSI2, [13](#)
 - CTE_SLAVE_EXTERNAL, [13](#)
 - Cte_TableUsageType, [14](#)
 - CTE_TOGGLE_DONT_CARE, [15](#)
 - CTE_TOGGLE_FLIP, [15](#)
 - CTE_TOGGLE_SET_TO_HIGH, [15](#)
 - CTE_TOGGLE_SET_TO_LOW, [15](#)
 - Cte_ToggleOutStatesType, [15](#)
 - CTE_TWO_TABLES, [14](#)
 - Std_ReturnType, [9](#)
- CTE AUTOSAR Driver Functions, [15](#)
 - Cte_GetLutChecksum, [16](#)
 - Cte_Restart, [16](#)
 - Cte_RfsGenerate, [16](#)
 - Cte_Setup, [16](#)
 - Cte_Start, [17](#)
 - Cte_Stop, [17](#)
 - Cte_UpdateTables, [17](#)
- Cte_ActionType, [19](#)
 - cteOutputSignalType, [19](#)
 - outputSignal, [19](#)
- CTE_CLOCK_ACTIVE
 - CTE AUTOSAR Driver Data Types, [9](#)
- CTE_CLOCK_ACTIVE_SYNC
 - CTE AUTOSAR Driver Data Types, [9](#)
- CTE_CLOCK_SET_TO_HIGH
 - CTE AUTOSAR Driver Data Types, [9](#)
- CTE_CLOCK_SET_TO_LOW
 - CTE AUTOSAR Driver Data Types, [9](#)

- CTE AUTOSAR Driver Data Types, 9
- Cte_ClockOutStatesType
 - CTE AUTOSAR Driver Data Types, 9
- CTE_CSI2_UNIT_0
 - CTE AUTOSAR Driver Data Types, 10
- CTE_CSI2_UNIT_1
 - CTE AUTOSAR Driver Data Types, 10
- CTE_CSI2_UNIT_MAX
 - CTE AUTOSAR Driver Data Types, 10
- CTE_CSI2_VC_0
 - CTE AUTOSAR Driver Data Types, 10
- CTE_CSI2_VC_1
 - CTE AUTOSAR Driver Data Types, 10
- CTE_CSI2_VC_2
 - CTE AUTOSAR Driver Data Types, 10
- CTE_CSI2_VC_3
 - CTE AUTOSAR Driver Data Types, 10
- CTE_CSI2_VC_MAX
 - CTE AUTOSAR Driver Data Types, 10
- Cte_Csi2UnitType
 - CTE AUTOSAR Driver Data Types, 10
- Cte_Csi2VcType
 - CTE AUTOSAR Driver Data Types, 10
- CTE_E_HW_ERROR
 - CTE AUTOSAR Driver Constants, 6
- CTE_E_PARAM_HANDLE
 - CTE AUTOSAR Driver Constants, 6
- CTE_E_PARAM_POINTER
 - CTE AUTOSAR Driver Constants, 6
- CTE_E_PARAM_VALUE
 - CTE AUTOSAR Driver Constants, 6
- CTE_E_SETUP_FAILED
 - CTE AUTOSAR Driver Constants, 7
- CTE_E_WRONG_STATE
 - CTE AUTOSAR Driver Constants, 7
- Cte_GetLutChecksum
 - CTE AUTOSAR Driver Functions, 16
- CTE_IRQ_RCS
 - CTE AUTOSAR Driver Data Types, 12
- CTE_IRQ_RFS
 - CTE AUTOSAR Driver Data Types, 12
- CTE_IRQ_TABLE_EXEC_END
 - CTE AUTOSAR Driver Data Types, 12
- CTE_IRQ_TT0_END
 - CTE AUTOSAR Driver Data Types, 12
- CTE_IRQ_TT0_START
 - CTE AUTOSAR Driver Data Types, 12
- CTE_IRQ_TT1_END
 - CTE AUTOSAR Driver Data Types, 12
- CTE_IRQ_TT1_START
 - CTE AUTOSAR Driver Data Types, 12
- Cte_IrqDefinitionType
 - CTE AUTOSAR Driver Data Types, 10
- Cte_IsrCbType
 - CTE AUTOSAR Driver Data Types, 9
- CTE_LOGIC_SET_TO_HIGH
 - CTE AUTOSAR Driver Data Types, 12
- CTE_LOGIC_SET_TO_HIGH_Z
 - CTE AUTOSAR Driver Data Types, 12
- CTE_LOGIC_SET_TO_LOW
 - CTE AUTOSAR Driver Data Types, 12
- CTE_LOGIC_UNCHANGED
 - CTE AUTOSAR Driver Data Types, 12
- Cte_LogicOutStatesType
 - CTE AUTOSAR Driver Data Types, 12
- CTE_MASTER
 - CTE AUTOSAR Driver Data Types, 13
- CTE_MODE_MAX
 - CTE AUTOSAR Driver Data Types, 13
- Cte_ModeDefinitionType, 20
 - cteWorkingParam0, 20
 - cteWorkingParam1, 20
 - workingMode, 20
- Cte_ModeType
 - CTE AUTOSAR Driver Data Types, 12
- CTE_ONE_TABLE
 - CTE AUTOSAR Driver Data Types, 14
- CTE_OUT_CLOCK
 - CTE AUTOSAR Driver Data Types, 14
- CTE_OUT_HIZ
 - CTE AUTOSAR Driver Data Types, 14
- CTE_OUT_LOGIC
 - CTE AUTOSAR Driver Data Types, 14
- CTE_OUT_MAX
 - CTE AUTOSAR Driver Data Types, 14
- CTE_OUT_TOGGLE
 - CTE AUTOSAR Driver Data Types, 14
- CTE_OUTPUT_CTEP_0
 - CTE AUTOSAR Driver Data Types, 13
- CTE_OUTPUT_CTEP_1
 - CTE AUTOSAR Driver Data Types, 13
- CTE_OUTPUT_CTEP_2
 - CTE AUTOSAR Driver Data Types, 13
- CTE_OUTPUT_CTEP_3
 - CTE AUTOSAR Driver Data Types, 13
- CTE_OUTPUT_CTEP_4
 - CTE AUTOSAR Driver Data Types, 13
- CTE_OUTPUT_CTEP_5
 - CTE AUTOSAR Driver Data Types, 13
- CTE_OUTPUT_CTEP_6
 - CTE AUTOSAR Driver Data Types, 13
- CTE_OUTPUT_CTEP_7
 - CTE AUTOSAR Driver Data Types, 13
- CTE_OUTPUT_FLEX_0
 - CTE AUTOSAR Driver Data Types, 13
- CTE_OUTPUT_FLEX_1
 - CTE AUTOSAR Driver Data Types, 14
- CTE_OUTPUT_MAX
 - CTE AUTOSAR Driver Data Types, 14
- CTE_OUTPUT_SPT_0
 - CTE AUTOSAR Driver Data Types, 13
- CTE_OUTPUT_SPT_1
 - CTE AUTOSAR Driver Data Types, 13
- CTE_OUTPUT_SPT_2
 - CTE AUTOSAR Driver Data Types, 13
- CTE_OUTPUT_SPT_3
 - CTE AUTOSAR Driver Data Types, 13

- CTE AUTOSAR Driver Data Types, 13
- CTE_OUTPUT_SPT_RCS
 - CTE AUTOSAR Driver Data Types, 13
- CTE_OUTPUT_SPT_RFS
 - CTE AUTOSAR Driver Data Types, 13
- Cte_OutputSignalType, 21
 - newClockState, 21
 - newLogicState, 21
 - newToggleState, 21
- Cte_OutputType
 - CTE AUTOSAR Driver Data Types, 13
- Cte_OutputTypeType
 - CTE AUTOSAR Driver Data Types, 14
- Cte_Restart
 - CTE AUTOSAR Driver Functions, 16
- Cte_RfsGenerate
 - CTE AUTOSAR Driver Functions, 16
- Cte_Setup
 - CTE AUTOSAR Driver Functions, 16
- Cte_SetupParamsType, 21
 - cteClockFreq, 22
 - ctelrqEvents, 22
 - cteMode, 22
 - irqExecCore, 22
 - irqPriority, 23
 - pCteCallback, 23
 - repeatCount, 23
 - signalDef0Ptr, 23
 - signalDef1Ptr, 23
 - timeTable0Ptr, 23
 - timeTable1Ptr, 24
- Cte_SingleOutputDefType, 24
 - clockPeriod, 24
 - outputSignal, 24
 - signalType, 25
- CTE_SLAVE_CSI2
 - CTE AUTOSAR Driver Data Types, 13
- CTE_SLAVE_EXTERNAL
 - CTE AUTOSAR Driver Data Types, 13
- Cte_Start
 - CTE AUTOSAR Driver Functions, 17
- Cte_Stop
 - CTE AUTOSAR Driver Functions, 17
- Cte_TableUsageType
 - CTE AUTOSAR Driver Data Types, 14
- Cte_TimeTableDefType, 25
 - eventsPtr, 25
 - tableLength, 25
 - tableTimeExecLimit, 26
- Cte_TimingEventType, 26
 - absTime, 26
 - eventActionsPtr, 26
- CTE_TOGGLE_DONT_CARE
 - CTE AUTOSAR Driver Data Types, 15
- CTE_TOGGLE_FLIP
 - CTE AUTOSAR Driver Data Types, 15
- CTE_TOGGLE_SET_TO_HIGH
 - CTE AUTOSAR Driver Data Types, 15
- CTE_TOGGLE_SET_TO_LOW
 - CTE AUTOSAR Driver Data Types, 15
- Cte_ToggleOutStatesType
 - CTE AUTOSAR Driver Data Types, 15
- CTE_TWO_TABLES
 - CTE AUTOSAR Driver Data Types, 14
- Cte_UpdateTables
 - CTE AUTOSAR Driver Functions, 17
- Cte_WorkingParam0, 27
 - cteCsi2Unit, 27
 - cteInternalRfsDelay, 27
- Cte_WorkingParam1, 28
 - cteCsi2Vc, 28
 - cteInternalRcsDelay, 28
- cteClockFreq
 - Cte_SetupParamsType, 22
- cteCsi2Unit
 - Cte_WorkingParam0, 27
- cteCsi2Vc
 - Cte_WorkingParam1, 28
- cteInternalRcsDelay
 - Cte_WorkingParam1, 28
- cteInternalRfsDelay
 - Cte_WorkingParam0, 27
- ctelrqEvents
 - Cte_SetupParamsType, 22
- cteMode
 - Cte_SetupParamsType, 22
- cteOutputSignalType
 - Cte_ActionType, 19
- cteWorkingParam0
 - Cte_ModeDefinitionType, 20
- cteWorkingParam1
 - Cte_ModeDefinitionType, 20
- eventActionsPtr
 - Cte_TimingEventType, 26
- eventsPtr
 - Cte_TimeTableDefType, 25
- irqExecCore
 - Cte_SetupParamsType, 22
- irqPriority
 - Cte_SetupParamsType, 23
- newClockState
 - Cte_OutputSignalType, 21
- newLogicState
 - Cte_OutputSignalType, 21
- newToggleState
 - Cte_OutputSignalType, 21
- outputSignal
 - Cte_ActionType, 19
 - Cte_SingleOutputDefType, 24
- pCteCallback
 - Cte_SetupParamsType, 23
- repeatCount

- Cte_SetupParamsType, [23](#)
- signalDef0Ptr
 - Cte_SetupParamsType, [23](#)
- signalDef1Ptr
 - Cte_SetupParamsType, [23](#)
- signalType
 - Cte_SingleOutputDefType, [25](#)
- Std_ReturnType
 - CTE AUTOSAR Driver Data Types, [9](#)
- tableLength
 - Cte_TimeTableDefType, [25](#)
- tableTimeExecLimit
 - Cte_TimeTableDefType, [26](#)
- timeTable0Ptr
 - Cte_SetupParamsType, [23](#)
- timeTable1Ptr
 - Cte_SetupParamsType, [24](#)
- workingMode
 - Cte_ModeDefinitionType, [20](#)

How to Reach Us:**Home Page:**nxp.com**Web Support:**nxp.com/support

Information in this document is provided solely to enable system and software implementers to use NXP products. There are no express or implied copyright licenses granted hereunder to design or fabricate any integrated circuits based on the information in this document. NXP reserves the right to make changes without further notice to any products herein.

NXP makes no warranty, representation, or guarantee regarding the suitability of its products for any particular purpose, nor does NXP assume any liability arising out of the application or use of any product or circuit, and specifically disclaims any and all liability, including without limitation consequential or incidental damages. "Typical" parameters that may be provided in NXP data sheets and/or specifications can and do vary in different applications, and actual performance may vary over time. All operating parameters, including "typicals," must be validated for each customer application by customer's technical experts. NXP does not convey any license under its patent rights nor the rights of others. NXP sells products pursuant to standard terms and conditions of sale, which can be found at the following address: nxp.com/SalesTermsandConditions.

NXP, the NXP logo, NXP SECURE CONNECTIONS FOR A SMARTER WORLD, COOLFLUX, EMBRACE, GREENCHIP, HITAG, I2C BUS, ICODE, JCOP, LIFE VIBES, MIFARE, MIFARE CLASSIC, MIFARE DESFire, MIFARE PLUS, MIFARE FLEX, MANTIS, MIFARE ULTRALIGHT, MIFARE4MOBILE, MIGLO, NTAG, ROADLINK, SMARTLX, SMARTMX, STARPLUG, TOPFET, TRENCHMOS, UCODE, Freescale, the Freescale logo, AltiVec, C-5, CodeTEST, CodeWarrior, ColdFire, ColdFire+, C-Ware, the Energy Efficient Solutions logo, Kinetis, Layerscape, MagniV, mobileGT, PEG, PowerQUICC, Processor Expert, QorIQ, QorIQ Converge, Ready Play, SafeAssure, the SafeAssure logo, StarCore, Symphony, VortiQa, Vybrid, Airfast, BeeKit, BeeStack, CoreNet, Flexis, MXC, Platform in a Package, QUICC Engine, SMARTMOS, Tower, TurboLink, and UMEMS are trademarks of NXP B.V. All other product or service names are the property of their respective owners. ARM, AMBA, ARM Powered, Artisan, Cortex, Jazelle, Keil, SecurCore, Thumb, TrustZone, and μ Vision are registered trademarks of ARM Limited (or its subsidiaries) in the EU and/or elsewhere. ARM7, ARM9, ARM11, big.LITTLE, CoreLink, CoreSight, DesignStart, Mali, mbed, NEON, POP, Sensinode, Socrates, ULINK and Versatile are trademarks of ARM Limited (or its subsidiaries) in the EU and/or elsewhere. All rights reserved. Oracle and Java are registered trademarks of Oracle and/or its affiliates. The Power Architecture and Power.org word marks and the Power and Power.org logos and related marks are trademarks and service marks licensed by Power.org.

© 2024 NXP B.V.

